

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Engineering and Technology

Devang Patel Institute of Advance Technology & Research

Department Of Computer Science & Engineering

A.Y. 2026-2027[Odd]

Lab Manual

CEUE203: Object Oriented Programming.

Name: Vaidehi Parmar

ID: 25DCS073

PRACTICAL 4

PART A:

1. Password strength (PasswordChecker.java + Driver.java)

(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.

(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4).

(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.

Expected output: "abc" → Weak; "Abcd1234!" → Strong.

Solution:

PasswordChecker.java

```
public class PasswordChecker {
    public static boolean lengthCheck(String pw) {
        return pw.length() >= 8;
    }

    public static boolean uppercaseCheck(String pw) {
        return pw.matches(regex: ".*[A-Z].*");
    }

    public static boolean digitCheck(String pw) {
        return pw.matches(regex: ".*\\d.*");
    }

    public static boolean specialCheck(String pw) {
        return pw.matches(regex: ".*[^a-zA-Z0-9].*");
    }

    public static String strength(String pw) {
        int count = 0;

        if (lengthCheck(pw)) count++;
        if (uppercaseCheck(pw)) count++;
        if (digitCheck(pw)) count++;
        if (specialCheck(pw)) count++;

        if (count <= 1) {
            return "Weak";
        } else if (count <= 3) {
            return "Medium";
        } else {
            return "Strong";
        }
    }
```

Driver4.java

```java
public class Driver4 {
    Run | Debug
    public static void main(String[] args) {
        String[] passwords = {
            "abc",
            "abcdefgh",
            "Abcdefgh",
            "Abcd1234",
            "Abcd1234!"
        };

        for (String pw : passwords) {
            System.out.println("Password: " + pw);
            System.out.println("Length >= 8: " + PasswordChecker.lengthCheck(pw));
            System.out.println("Uppercase: " + PasswordChecker.uppercaseCheck(pw));
            System.out.println("Digit: " + PasswordChecker.digitCheck(pw));
            System.out.println("Special character: " + PasswordChecker.specialCheck(pw));
            System.out.println("Strength: " + PasswordChecker.strength(pw));
            System.out.println();
        }
    }
}
```

Output:

```
Password: abc
Length >= 8: false
Uppercase: false
Digit: false
Special character: false
Strength: Weak

Password: abcdefgh
Length >= 8: true
Uppercase: false
Digit: false
Special character: false
Strength: Weak

Password: Abcdefgh
Length >= 8: true
Uppercase: true
Digit: false
Special character: false
Strength: Medium

Password: Abcd1234
Length >= 8: true
Uppercase: true
Digit: true
Special character: false
Strength: Medium

Password: Abcd1234!
Length >= 8: true
Uppercase: true
Digit: true
Special character: true
Strength: Strong
```

2. Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder.

(e) Print “Matches: N” and the built report.

Expected output: keyword “hello” over 3 lines (1 malformed) → “Matches: 1” and the matching line.

Solution:

ChatFilter.java

```java
public class ChatFilter {
    public static boolean containsKeyword(String message, String keyword) {
        return message.toLowerCase().contains(keyword.toLowerCase());
    }
}
```

Driver5.java

```java
import java.util.Scanner;

public class Driver5 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] logs = {
            "10:05 alice Hello there",
            "10:10 bob How are you?",
            "10:15 charlie",
            "10:20 david Good morning"
        };

        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();

        int count = 0;
        StringBuilder report = new StringBuilder();

        for (String line : logs) {
            String[] parts = line.split(" ", 3);

            if (parts.length < 3) {
                continue;
            }

            String time = parts[0];
            String user = parts[1];
            String message = parts[2];

            if (ChatFilter.containsKeyword(message, keyword)) {
                count++;
                report.append(time)
                      .append(" ")
                      .append(user)
                      .append(": ")
                      .append(message)
                      .append("\n");
            }
        }

        System.out.println("Matches: " + count);
        System.out.print(report);

        sc.close();
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver5.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver5
Enter keyword: hello
Matches: 1
10:05 alice: Hello there
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver5.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver5
Enter keyword: good
Matches: 1
10:20 david: Good morning
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver5.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver5
Enter keyword: xyz
Matches: 0
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3>
```

3. Template filler (TemplateFiller.java + Driver.java)

(a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date} is not supplied).

(b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template.

(c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found.

(d) Print the filled template.

Expected output: the unsupplied {date} becomes [?].

Solution:

TemplateFiller.java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;
public class TemplateFiller {
    public static String fill(String template, String[] names, String[] values) {
        Pattern pattern = Pattern.compile(regex: "\\{(\\w+)\\}");
        Matcher matcher = pattern.matcher(template);
        StringBuilder result = new StringBuilder();
        int lastEnd = 0;

        while (matcher.find()) {
            result.append(template, lastEnd, matcher.start());

            String placeholder = matcher.group(group: 1);
            String replacement = "[?]";

            for (int i = 0; i < names.length; i++) {
                if (names[i].equals(placeholder)) {
                    replacement = values[i];
                    break;
                }
            }

            result.append(replacement);
            lastEnd = matcher.end();
        }
        result.append(template, lastEnd, template.length());
        return result.toString();
    }
}
```

Driver6.java

```java
public class Driver6 {
    Run | Debug
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";

        String[] names = {"name", "id"};
        String[] values = {"Riya", "A07"};

        String result = TemplateFiller.fill(template, names, values);

        System.out.println(result);
    }
}
```

Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> javac Driver6.java
PS C:\Users\parma\OneDrive\Documents\OOPs-sem3> java Driver6
Dear Riya, order A07 ships [?].
```